

System Requirements Specification (FR & NFR)

Sep 29, 2026 · @Sanjeevan

Introduction

This specification merges the four separately-written role documents (Store Manager, Dispatcher, Loader, Driver) for **Waypoint**, the delivery-coordination system built for Tech-Triathlon 2026, into one consistent functional and non-functional requirements baseline.

Waypoint coordinates deliveries from two depots (Peliyagoda, Kandy) to 120 outlets across three brands with very different rhythms: Fresh (80 outlets, near-daily chilled + dry goods, must arrive before 8 AM), Style (25 outlets, weekly, half inside mall access windows), and Tech (15 outlets, as-needed, high-value/fragile single items). A fleet of 60 vehicles runs up to two trips a day.

The challenge booklet names four problems the system must fix: planning is fragmented (orders arrive by phone or message and are re-entered by hand); deferrals lack a clear record (the same outlet can go unserved on consecutive runs, unnoticed); communication doesn't support feedback (no reliable proof of delivery or issue record); and demand is difficult to anticipate. Every requirement below traces back to one of these problems, to an explicit operating constraint, or is flagged as a team-proposed addition that closes a gap the brief implies but never states.

Four roles carry the workflow from order to resolution: the **Store Manager** places orders and confirms receipt, the **Dispatcher** turns the order queue into a feasible, constraint-respecting delivery plan, the **Loader** loads vehicles against that plan and flags shortfalls before departure, and the **Driver** executes the trip and records what actually happened. This document first sets out the requirements that cut across all four roles, then documents each role in turn, and closes with the additional features and shared data model that tie the roles together.

System Actors

Role	Device / environment	Primary responsibility	Key constraint
Store Manager	Outlet counter, desktop or phone	Places orders, tracks arrival, confirms receipt, reports issues	Zero-training retail staff; account scoped to one outlet_id
Dispatcher	Planning office (Peliyagoda/Kandy), desktop	Closes the order queue, allocates 60 vehicles across 120 outlets, defers over-capacity orders with a reason, publishes and monitors the plan, forecasts future capacity	Every decision must be explainable, auditable and consistent regardless of who is on shift
Loader	Depot dock, shared tablet/terminal	Loads vehicles against the published stop sequence, flags	Shared device, no personal login; must

Role	Device / environment	Primary responsibility	Key constraint
		missing/damaged items before departure	work fast, one-handed, under time pressure
Driver	On the road, personal phone	Executes the assigned trip, records delivery outcome and proof, reports issues, works through connectivity gaps	Safety-first minimal interaction while moving; must degrade gracefully offline

The four roles share one workflow: an order placed by a Store Manager is allocated and published by the Dispatcher, loaded and flagged by the Loader, delivered and recorded by the Driver, and finally confirmed (or disputed) by the Store Manager — closing the loop back to the Dispatcher.

Overall Functional Requirements (System-Wide)

These requirements are not owned by a single role — they hold the four role-specific portals together as one consistent system.

ID	Requirement	Spans
OFR1	All confirmed orders arrive in one Dispatcher queue, eliminating phone/message re-entry into a spreadsheet.	Store → Dispatcher
OFR2	One shared status vocabulary is used everywhere: Submitted → Confirmed/Received → Allocated → Loaded → Out for delivery/Departed → Delivered, or Deferred, plus an issue-case lifecycle (Reported → Acknowledged → Under review → Resolved).	All roles
OFR3	Each role gets one adaptive portal, not brand-forked apps: Fresh/Style/Tech differences are conditional content driven by outlet.brand and vehicle.temperature, never separate applications.	All roles
OFR4	Order confirmations, deferral notices with reason, arrival-time/ETA changes, loader shortfall flags, delivery outcomes and issue updates are	Store ↔ Dispatcher ↔ Loader ↔ Driver

ID	Requirement	Spans
	pushed automatically between roles (in-app plus SMS/push for time-critical items), never left to a phone call.	
OFR5	Every complaint becomes a tracked issue case with evidence, a responsible stage and a fair, auditable outcome. A three-way count comparison (loader-confirmed × driver proof-of-delivery × store received-count) locates which stage a shortfall happened at, so only verified, stage-attributed cases affect that stage's performance record (for example, only a verified driver-attributed case affects a driver's score).	Store, Loader, Driver, Dispatcher
OFR6	Deferred orders carry a mandatory reason code, are flagged automatically when an outlet is skipped on consecutive runs, and roll over to the next run's queue with priority preserved.	Dispatcher → Store
OFR7	Calendar data (festival, payday, monsoon, holiday flags) is shared across the ordering screen, the demand forecast and the capacity plan, so all three reflect the same known-future demand shifts.	Store, Dispatcher
OFR8	Outlet, vehicle, order and route identifiers match one shared master dataset exactly, with no reformatting, across all four role views.	All roles
OFR9	Every order, allocation, deferral, load confirmation, delivery and issue action is timestamped and recorded as part of one audit trail spanning all roles,	All roles

ID	Requirement	Spans
	supporting dispute resolution end to end.	

Overall Non-Functional Requirements (System-Wide)

Category	Requirement
Consistency	Status labels, colour tokens, icons and terminology ("Deferred", "Confirmed", "Allocated", "Loaded") are identical across all four role interfaces; the same event looks the same everywhere it appears.
Accessibility	Text contrast meets WCAG AA ($\geq 4.5:1$). Every status or category is conveyed by colour and icon and text label together — never colour alone.
Localization	The system supports Sinhala, Tamil and English, selectable per account; all times use Asia/Colombo and the 24-hour clock; dates display as "Wed 30 Sep".
Auditability	Every plan change, order, allocation, load confirmation, delivery, deferral and issue action is timestamped and immutable — corrections are added as new entries, never edited in place.
Role-scoped security	Each account or shared device sees only its own outlet, depot or assigned data; a Store Manager never sees other outlets or fleet data, a Loader never sees Dispatcher planning controls or the other depot, a Dispatcher is depot-scoped unless granted cross-depot access.
Resilience, tiered by role	Driver: full offline operation for core delivery work, with sync and conflict-flagging on reconnect (highest need — en route with a personal phone). Store Manager: drafts kept on-device in low-connectivity districts. Loader: brief dock network drops must not lose in-progress checklist state. Dispatcher: all plan state is server-persisted so a browser crash or refresh loses nothing.
No silent failure	Every submission — an order, a load confirmation, an issue report, a delivery record — ends in a clear success state with an ID, or a clear, retryable failure state. Nothing is ever silently accepted or silently lost.

Category	Requirement
Context-matched usability	Store Manager: plain language, zero training, desktop/phone parity. Loader: large touch targets, minimal typing, high contrast for a shared tablet in a noisy, variably-lit warehouse. Driver: a minimal, glanceable "Drive Mode" with detailed interaction available only after the vehicle has safely stopped. Dispatcher: a dense, keyboard-operable desktop planning view with shortcuts for high-frequency actions.
Performance	The assisted-allocation engine produces a full day's plan (up to 200 orders, 60 vehicles) within 30 seconds; the live board reflects a driver-recorded event within 10 seconds; UI transitions and filters complete within 1 second.
Scalability & maintainability	The allocation engine sits behind a replaceable internal API; deferral priority rules are stored as editable configuration, not hard-coded logic; the system supports adding a third depot without structural changes.

Store Manager

Works at the outlet counter on a desktop or phone; one account is tied to one outlet_id, which carries one brand (Fresh, Style or Tech). The portal is brand-aware, not brand-forked: six main pages (Dashboard, Place/Manage Order, Order Status, Receive, History & Issues, Settings) plus a Notifications panel adapt their content to the outlet's brand.

Functional requirements

ID	Requirement	Priority
FR1	Place an order before the 4 PM cutoff; the form adapts to brand (Fresh: dry + chilled tabs; Style: one weekly order; Tech: single-item form with value/fragility notes).	Must
FR2	Cutoff countdown shown persistently; after 16:00 a new order is clearly labelled for the next run.	Must
FR3	Immediate order confirmation with ID and timestamp; status then progresses Submitted → Received → Allocated → Out	Must

ID	Requirement	Priority
	for delivery → Delivered/Deferred.	
FR4	See the planned arrival time and delivery window once the Dispatcher publishes the plan.	Must
FR5	Proactive deferral notice with plain-language reason and new delivery date; a consecutive-skip warning if the outlet was also deferred last run.	Must
FR6	Confirm receipt: received in full / received short or damaged / (Tech) accept with reservation pending inspection.	Must
FR7	Report an issue: affected lines, issue type, quantity, optional note/photo; creates a tracked issue case.	Must
FR8	See the outlet's order and delivery history, with consecutive deferrals highlighted.	Must
FR9	Edit or cancel a submitted order until the 16:00 cutoff.	Should
FR10	Report a store closure or access change (mall event, roadworks, bay closed).	Should
FR11	See a plain-language live delivery status once the vehicle departs.	Should
FR12	See the resolution outcome of any reported issue (replacement, credit, or not upheld with a reason); confirm or reopen once.	Must
FR13	See an expected-shortfall flag before the truck arrives, if the Loader flagged a	Should

ID	Requirement	Priority
	missing/damaged item at departure.	
FR14	One notifications centre for every message to the store, each acknowledgeable.	Must

Non-functional requirements

ID	Requirement
NFR1	Zero-training usability: plain language, no fleet jargon, one short form per action.
NFR2	Desktop and phone parity — both must work equally well.
NFR3	Proactive notification (in-app plus SMS/push) for deferrals, arrival-time changes and expected shortfalls.
NFR4	No silent submission failure, especially near the 16:00 cutoff.
NFR5	Every status shown with colour, icon and text together; contrast $\geq 4.5:1$.
NFR6	Status names, colours and icons match the Dispatcher, Loader and Driver faces exactly.
NFR7	Every order, edit, receipt confirmation and issue report is timestamped and cannot be altered after submission.
NFR8	Outlet-scoped access only — never other outlets, vehicle capacity or dispatcher tools.
NFR9	Brand differences are conditional content on outlet.brand, not separate applications.
NFR10	Arrival times shown as planned time plus window, never false precision.
NFR11	Resilient to weak rural/hill-country connectivity — drafts kept on-device until reconnected.
NFR12	Sinhala, Tamil and English; Asia/Colombo, 24-hour clock.

Degradation is designed for explicitly: an **Order Deferred** screen (the direct fix for "no clear notice when an order is deferred") and a **submission-fails-near-cutoff** flow that saves the draft, retries automatically, and never reports a false success.

Dispatcher

Works from a large desktop screen at the Peliyagoda or Kandy planning office. The operational backbone of the daily cycle: closes the order queue, builds a constraint-respecting delivery plan for 60 vehicles and 120 outlets, decides and justifies every deferral, publishes to loaders/drivers/stores, monitors live progress, and plans future fleet capacity. This is the largest role by requirement count — **105 functional and 23 non-functional requirements (80 Must-have)** — grouped into 16 functional areas below.

Functional requirement areas

Area	Covers	Must-have highlights
App frame & navigation	Persistent Plan / Live Board / Deferral Log / Capacity Forecast navigation; depot and date context; operating-day calendar	Only operating days (Mon–Sat) selectable
Order intake & queue	Collecting all confirmed orders into one queue, enforcing the 4 PM cutoff	Cutoff enforced; late orders auto-routed to the next run; unusual-quantity flags
Planning & allocation	Assigning every in-scope order to one vehicle and trip	Assisted planning (system suggests, dispatcher approves) is the required default; manual overrides re-check constraints instantly
Constraint enforcement	Weight, volume, temperature, van-only outlets, mall windows, delivery windows, fuel quota, depot/brand/district matching, whole-order integrity	Publishing is blocked while any hard constraint is violated
Capacity shortfall & deferrals	Identifying orders that don't fit, requiring a reason for every deferral, protecting against consecutive skips	No order may be deferred without a recorded reason; consecutive-skip protection is mandatory
Publish & hand-off	Pushing the finished plan to loaders, drivers and stores; re-publishing on change	Publishing is blocked on unresolved hard-constraint violations; every publish/re-publish is versioned
Live board	Real-time progress per route, sync status, planned-vs-actual timing	Reflects driver-recorded outcomes immediately; legend never colour-only
Issues & alerts	A single "needs attention" feed for every operational exception	Critical alerts (breakdown, vehicle loss) are visually distinct and resolved with a mandatory note

Area	Covers	Must-have highlights
Communication	Automated ETA, deferral and shortfall notices to loaders/drivers/stores	All messages sent and logged in-system, replacing verbal instructions
Deferral log & outlet history	A searchable, exportable record of every deferral and its outcome	Consecutive-deferral outlets highlighted automatically
Capacity forecast	Weekly demand vs. available fleet by depot and brand, with calendar markers	Shortfall weeks and recommended extra vehicles are surfaced
Reference data management	Outlets, vehicles, operating calendar, vehicle workshop status	All 120 outlets and 60 vehicles visible with their governing attributes
Offline records (dispatcher side)	Merging driver-side offline records safely on reconnect	Conflicting server/offline state is surfaced, never silently overwritten
Audit, records & disputes	Full audit trail; loader–driver–store count comparison for delivered orders	Audit trail is immutable
Shift continuity	Handover notes, multi-dispatcher trip locking	— (Should-have)
Degradation scenarios	Peak-day overload, vehicle breakdown, workshop loss, flooding, extended offline, office power loss	Peak-day overload flow (priority rules, swap suggestions, bulk defer) is required

Non-functional requirements

Category	Requirement
Performance	Full-day plan for 200 orders/60 vehicles within 30s; manual-override constraint re-check within 3s; live board refresh within 10s; UI transitions within 1s; forecast charts load within 5s.
Reliability & data integrity	All plan state server-persisted on every save; offline merges use device timestamps and flag conflicts; the audit trail is immutable; proof-of-delivery assets retained ≥ 90 days.
Usability & accessibility	Desktop-first, data-dense, no horizontal scrolling; WCAG AA contrast; status never colour-only; every control keyboard-operable with shortcuts for high-frequency actions; every automated

Category	Requirement
	decision explained in one plain-language line; explicit empty/loading states.
Consistency & terminology	Status labels, colours, icons and identifiers identical across all four roles and matching the shared datasets exactly.
Security & access	Only authenticated Dispatcher accounts may plan, publish, defer or change vehicle status; depot-scoped by default; sessions time out on inactivity.
Internationalisation	Sinhala, Tamil, English; recipient-preferred language on outgoing messages.
Scalability & maintainability	Allocation engine replaceable behind an internal API; deferral priority rules are editable configuration; a third depot can be added without structural change.

Driver

On the road with a personal phone; connectivity may drop, especially in hill country and the Kandy corridor. Executes the dispatcher's assigned trip, follows the planned stop sequence, records each delivery outcome with proof, and reports issues — feeding real events back into the system.

Functional requirements

ID	Requirement	Priority
DR01	View the assigned trip and its stops.	Must
DR02	Start the selected trip (an explicit state change into "active").	Must
DR03	A minimal "Drive Mode" view while travelling — next stop, ETA, window — kept deliberately glanceable.	Must
DR04	Show next stop, planned arrival/ETA and delivery window.	Must
DR05	Record each delivery stop's outcome.	Must

ID	Requirement	Priority
DR06	Capture proof of delivery (photo, signature or recipient name).	Must
DR07	Report delivery issues (missing, damaged, wrong item, late, other).	Must
DR08	Operate offline for core delivery work when connectivity drops.	Must
DR09	Synchronize offline records automatically when connectivity returns, flagging any conflict with the server state.	Must
DR10	Show sync status and allow retry of a failed synchronization.	Should
DR11	Pause and resume an active trip for a legitimate temporary stop, without ending it.	Should
DR12	Receive dispatch updates (route/stop changes), reviewable once safely stopped rather than mid-drive.	Should
DR13	Show completed-stop / remaining-stop progress for the run.	Should
DR14	End the trip and show its final status.	Should

Non-functional requirements

Category	Requirement
Usability	Simple, low-step interaction; key information must be understandable at a glance.
Safety	Detailed delivery actions are only available after the Driver indicates they have safely stopped — Drive Mode stays minimal while moving.
Performance	Opening the current stop and recording an outcome must be fast.
Reliability	Offline delivery records and proof must never be lost when connectivity drops.

Category	Requirement
Offline availability	Core delivery execution must keep working with no connection at all — the highest offline priority of any role.
Data integrity	Outcome, proof, issue and timestamp stay linked to the correct delivery.
Security / access	The Driver sees only their own trip, delivery and account information.
Responsive design	Phone-first; next stop, ETA/window, status and sync state are highly visible on a small screen.
Consistency	Status terms and delivery states match the Dispatcher, Loader and Store Manager faces exactly.

The design principle carried through every screen is **Drive → Stop → Interact → Drive**: safety-relevant interaction only happens once the vehicle is stopped. The one required degradation scenario is connectivity loss during an active trip — the Driver keeps working from cached trip data, records outcomes locally, and syncs automatically on reconnection, with failed records left retryable rather than lost.

Loader

Works at the Peliyagoda or Kandy warehouse dock on a shared tablet or terminal — no personal login. Loads vehicles against the Dispatcher's published stop sequence and flags missing or damaged items before a vehicle leaves, the first problem-report in the pipeline. Role begins once goods are already staged at the depot; inbound receiving is out of scope.

Functional requirements

ID	Requirement	Priority
FR1	View all trips assigned to the depot for the day; Fresh (pre-8 AM) trips are visually distinct from more flexible Style/Tech trips.	Must
FR2	View the planned stop sequence per trip — generally the reverse of delivery order, for last-in/first-out unloading.	Must
FR3	View order details per stop: outlet, brand, weight/volume, temperature requirement, dock type.	Must

ID	Requirement	Priority
FR4	Confirm individual orders/items as physically loaded (checklist).	Must
FR5	Flag missing or damaged items with a type, optional note/photo, reaching the Dispatcher in near real time.	Must
FR6	Receive live plan updates — the direct fix for outdated printed loading lists.	Must
FR7	Confirm a trip fully loaded, the hand-off that makes it visible to the Driver.	Must
FR8	Complete a trip's load even with some items flagged missing/damaged — a documented shortfall, never a hard block.	Must
FR9	Distinguish Trip 1 from Trip 2 for a vehicle running twice in a day.	Could
FR10	Un-check a mis-tapped item or withdraw a flag any time before the trip is confirmed loaded.	Must
FR11	Filter and sort the trip queue by urgency, vehicle or loading status.	Must
FR12	Alert the Loader if the Dispatcher reassigns or modifies a trip after it was already confirmed loaded, so it can be reopened and re-verified.	Must

Non-functional requirements

ID	Requirement
NFR1	Usable on a shared device — no persistent personal login, large touch targets, minimal typing.
NFR2	Speed / low friction — every action completable in a few taps, especially before Fresh's 8 AM departures.

ID	Requirement
NFR3	Real-time / near-real-time data sync, so the Loader is never working from a stale plan.
NFR4	Legible in a noisy, variably-lit warehouse: high contrast, large text, clear icons over dense tables.
NFR5	The flag mechanism is highly reliable, with no silent failures and clear on-screen confirmation.
NFR6	Every load confirmation, flag and correction is timestamped and recorded.
NFR7	Resilient to brief interruption — in-progress checklist state survives a locked tablet or a shift change.
NFR8	Shares visual language (colours, icons, status) with the Dispatcher and Driver faces — judged under cross-role coherence.
NFR9	Tablet/mobile-first layout — vertical, thumb-reachable, no wide desktop-style tables.
NFR10	Tolerates brief connectivity hiccups at the dock without losing checklist state (lower priority than the Driver's full offline need).
NFR11	Role-scoped: the shared terminal exposes only its own depot's trips, never Dispatcher controls or the other depot.
NFR12	Every status conveyed by icon or text label as well as colour.

Additional Features (Cross-Role, Beyond the Explicit Brief)

Each role document proposed features that go beyond what the challenge booklet states outright, but close a gap the booklet implies. Kept here together because none belongs to one role alone — each feeds at least one other role.

Feature	Closes	Feeds
Issue case & fair-attribution accountability	"Disputes depend on memory"; unfair blame between stages	Store, Loader, Driver, Dispatcher — the central cross-role mechanism
Missing-order reminder	A busy Store Manager missing the 4 PM cutoff by accident	Dispatcher (fewer missing orders, fewer urgent exceptions)

Feature	Closes	Feeds
Calendar-aware order guidance	"Demand is difficult to anticipate", addressed at the point orders are placed	Dispatcher (earlier, more realistic ordering ahead of peaks)
Delivery feedback for driver service quality	A trustworthy signal into driver scoring from the receiving end	Driver points system, Dispatcher service reports
Outlet knowledge contribution (gate codes, unloading notes)	Operational knowledge living only in one dispatcher's head	Driver stop details, Dispatcher outlet knowledge cards
Receiving hours & delivery-window management	Deliveries planned for times no one is there to receive them	Dispatcher plan validity
Critical-need signal (lowest priority, may be cut)	An urgent single-item need, e.g. "out of infant formula"	Dispatcher priority rules only — never a binding allocation
Swap suggestions & knock-on-effect preview	Manual deferral decisions made without seeing the consequence	Dispatcher deferral workflow
Predicted service time / lateness per stop	Planning against a fixed time instead of a realistic one	Dispatcher trip planning, Live Board
Brand service-share fairness summary	One brand being quietly deprioritised every day	Dispatcher capacity decisions
Capacity what-if tool	Fleet-hiring decisions made without a testable scenario	Dispatcher capacity forecast
Partial loads with documented shortfall	A trip blocked entirely by one missing item	Loader, Driver, Store (expected-shortfall flag)
Post-confirmation change alert	A race condition between Loader hand-off and a late Dispatcher edit	Loader, Dispatcher
Pause / resume an active trip	A driver's legitimate short stop ending the whole trip record	Driver

Shared Data Entities

All four roles read and write the same underlying records: **Outlet** (brand, district, depot, dock type, mall/delivery window), **Vehicle** (type, temperature capability, weight/volume limits, fuel quota, depot), **Order/Delivery** (outlet, brand, type, temperature requirement, units/weight/volume, status history), **Trip/Route** (vehicle, stop sequence, planned times), **Delivery Outcome** and **Proof of Delivery** (photo, signature, receiver name, timestamp), **Issue Case** (lifecycle, attributed stage, evidence, outcome), and **Sync Record** (for offline reconciliation). Keeping one definition of each entity, referenced identically by every role's screens, is what makes the four separate documents behave as a single consistent system rather than four separate apps that happen to share a name.

End-to-End Flow

Store places an order → Dispatcher allocates it to a vehicle and trip and publishes the plan (or defers it, with a reason, back to the Store) → Loader loads the vehicle against the published sequence and flags any shortfall → Driver executes the trip and records the delivery outcome with proof → Store confirms receipt or reports an issue → any issue is resolved through the fair-attribution process, feeding outlet service-health metrics and driver scoring back into the Dispatcher's view. This loop, and the requirement that every step in it is visible to the roles it affects, is the throughline connecting all four documents.